AWS EC2 Start/Stop Automation Using Lambda

Objective

Build an automated solution where:

1. A Lambda function starts EC2 instances
2. A Lambda function stops EC2 instances
3. Instance IDs are predefined in the function
4. (Optional) Can be triggered manually or via CloudWatch Scheduler

---

Architecture Overview

Lambda → EC2 API → Start / Stop Instances

(Optional) EventBridge Scheduler → Lambda → EC2

Prerequisites

- AWS Account
- IAM permissions to create Lambda and IAM roles
- Existing EC2 instances
- Instance IDs available

Step 1: Identify EC2 Instance IDs

1. Go to AWS Console → EC2
2. Select Instances
3. Copy the Instance IDs

Example:

- i-0123456789abcdef0
- i-0abcdef1234567890

Step 2: Create IAM Role for Lambda

1. Go to IAM → Roles
2. Click Create Role
3. Select AWS Service → Lambda
4. Attach:
    - AWSLambdaBasicExecutionRole
5. Create a custom policy with EC2 permissions

Custom EC2 Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances"
      ],
      "Resource": "*"
    }
  ]
}
```

Note: For stricter security, replace "*" with specific EC2 instance ARNs.

Name the role: EC2StartStopLambdaRole

Step 3: Create Lambda Function (Stop EC2)

1. Go to AWS Console → Lambda
2. Click Create Function
3. Choose Author from scratch
4. Function Name: StopEC2Instances
5. Runtime: Python 3.x
6. Execution Role: Use existing role → EC2StartStopLambdaRole
7. Click Create

Step 4: Add Stop EC2 Code

Replace default code with:

```
import boto3


def lambda_handler(event, context):
    region = 'us-east-1'  # Change region if required

    instance_ids = [
        'i-0123456789abcdef0',
        'i-0abcdef1234567890'
    ]

    ec2 = boto3.client('ec2', region_name=region)

    try:
        response = ec2.stop_instances(InstanceIds=instance_ids)
        print(f"Stopping instances: {instance_ids}")
        return response

    except Exception as e:
        print(f"Error stopping instances: {e}")
        raise e
```

Click Deploy.

Step 5: Create Lambda Function (Start EC2)

Repeat previous steps:

Function Name: StartEC2Instances Runtime: Python 3.x Role: EC2StartStopLambdaRole

---

Step 6: Add Start EC2 Code

```
import boto3


def lambda_handler(event, context):
    region = 'us-east-1'  # Change region if required

    instance_ids = [
        'i-0123456789abcdef0',
        'i-0abcdef1234567890'
    ]

    ec2 = boto3.client('ec2', region_name=region)

    try:
        response = ec2.start_instances(InstanceIds=instance_ids)
        print(f"Starting instances: {instance_ids}")
        return response

    except Exception as e:
        print(f"Error starting instances: {e}")
        raise e
```

Click Deploy.

Step 7: Test the Lambda Functions

1. Open StopEC2Instances
2. Click Test → Create test event (empty JSON {})
3. Run test → Verify EC2 instances move to “stopping” state

Repeat for StartEC2Instances.

Step 8: Automate Using EventBridge Scheduler

To automatically stop/start instances daily:

1. Go to EventBridge → Scheduler
2. Click Create Schedule

3. Choose:
    - Schedule type: Recurring
    - Cron expression (example below)

Example:

Stop at 10 PM daily:

```
cron(0 22 * * ? *)
```

Start at 8 AM daily:

```
cron(0 8 * * ? *)
```

4. Target → Lambda Function
5. Select corresponding function
6. Save

Production Improvements

1. Use Environment Variables Instead of Hardcoding

Add environment variables:

Key	Value
REGION	us-east-1
INSTANCE_IDS	i-0123...,i-0abc...

Then modify code:

```
import boto3
import os


def lambda_handler(event, context):
    region = os.environ['REGION']
    instance_ids = os.environ['INSTANCE_IDS'].split(',')

    ec2 = boto3.client('ec2', region_name=region)

    return ec2.stop_instances(InstanceIds=instance_ids)
```

2. Add Logging & Monitoring

- Monitor via CloudWatch Logs
- Create CloudWatch Alarms for Lambda failures

3. Restrict IAM Permissions

Instead of “Resource”: “*”, use specific EC2 ARNs.

Security Best Practices

- Use least privilege IAM policies
- Enable CloudTrail logging
- Restrict Lambda invocation permissions
- Use tagging to control which instances can be started/stopped

---

Final Validation Checklist

☐ EC2 instance IDs verified
☐ IAM role created with EC2 permissions
☐ Stop Lambda deployed
☐ Start Lambda deployed
☐ Manual test successful
☐ Scheduler configured (if required)

Result

You now have an automated EC2 start/stop solution using AWS Lambda. This can reduce costs by shutting down non-production environments outside business hours and starting them automatically when required.